

# Embeder Desktop Application Architecture

**Embeder** is an open-source, local-first AI copilot and desktop workspace engineered for embedded systems design, hardware prototyping, firmware compilation, and automated PCB generation.

## 1. High-Level Architecture

Embeder operates on a client-daemon paradigm designed to keep the user interface responsive while heavy hardware background tasks run locally.

| Layer                 | Technology Stack               | Primary Responsibility                                                                                           |
|-----------------------|--------------------------------|------------------------------------------------------------------------------------------------------------------|
| UI Shell              | Tauri / Rust + Web Frontend    | Native performance, low memory footprint, split-pane IDE interface, canvas viewer.                               |
| Desktop Engine / Core | Rust / Node.js Process Manager | Orchestrates background processes, manages BYOK API configurations, controls local LLM runtime links.            |
| MCP Orchestrator      | Model Context Protocol Client  | Routes model intents to specialized local MCP tool servers.                                                      |
| Toolchain Workers     | Isolated Local CLI Services    | Executes compilation (`avr-gcc`, `esp-idf`, `arm-none-eabi-gcc`), Renode simulations, and KiCad script routines. |

## 2. Key UI/UX Modules

- **Conversational Guidance & Reasoning Panel:** Displays structured multi-step engineering prompts, teaching paths, and interactive decision trees.
- **Unified Code & Firmware Editor:** Built-in code viewer with real-time compilation log streams and register state monitors.
- **Visual Hardware Canvas:** Renders interactive circuit schematics, pinout assignments, and component connections.
- **PCB Design Studio:** Displays programmatically generated KiCad layouts, netlists, and Gerber export settings.
- **Settings & BYOK Management:** Secure storage for user API keys (OpenAI, Anthropic, Gemini) and local LLM endpoints (Ollama, Llama.cpp).

## 3. Local Data & Workspace Architecture

`embeder-workspace/`

|                                |                                            |
|--------------------------------|--------------------------------------------|
| — project.json                 | # Global project metadata & MCU targets    |
| — src/                         | # C/C++ firmware source files              |
| — datasheets/<br>manuals       | # Local vector index of MCU pinouts &      |
| — simulation/<br>trace dumps   | # Renode scripts, virtual logic logs, wave |
| └─ pcb/<br>layout (.kicad_pcb) | # KiCad schematic files (.kicad_sch), PCB  |